

Processamento de Documentos

Bárbara Xavier Soares de Barros
Daniel Vitor Salviano Araújo
Felipe Fernandes de Bellis Ruas
Gustavo Augusto Santos Perdigão
Matheus Henrique Marques Canuto
Yago de Arruda Simões

Instituto de Ciências Exatas e de Informática – Pontifícia Universidade Católica de Minas Gerais

Curso de Sistemas de Informação – Contagem – MG – Brasil

Disciplina: Recuperação de Informação na Web e Redes Sociais

Professora: Jeanne Louize Emygdio

`barbara.barros.1426627@sga.pucminas.br`

`daniel.araujo.1439479@sga.pucminas.br`

`felipe.bellis@sga.pucminas.br`

`gasperdigao@sga.pucminas.br`

`matheus.canuto.1380848@sga.pucminas.br`

`yago.arruda@sga.pucminas.br`

Resumo

A recuperação de informação depende de etapas capazes de transformar documentos em representações adequadas para indexação, busca e ranqueamento. Nesse contexto, o processamento de documentos atua como uma fase essencial dos Sistemas de Recuperação de Informação, pois reduz a complexidade do texto integral e permite extrair termos representativos. Este artigo tem como objetivo analisar técnicas clássicas e recentes de processamento de documentos, com ênfase em análise léxica, remoção de *stopwords*, *stemming*, lematização, seleção de palavras-chave e estruturas de categorização. A metodologia adotada combina fundamentação teórica, revisão de quatro trabalhos relacionados publicados entre 2021 e 2026 e uma proposta experimental de implementação de um protótipo de pré-processamento textual em Python. Como resultado, apresenta-se um fluxo de processamento capaz de normalizar textos, remover ruídos, reduzir variações morfológicas e gerar termos de indexação. Conclui-se que o processamento de documentos influencia diretamente a qualidade da recuperação, pois afeta tanto a representação dos documentos quanto a correspondência

entre consultas e itens relevantes.

Palavras-chave: processamento de documentos; recuperação de informação; indexação textual; pré-processamento textual; *stemming*.

Abstract

Information retrieval depends on steps capable of transforming documents into suitable representations for indexing, search, and ranking. In this context, document processing is an essential phase of Information Retrieval Systems, as it reduces the complexity of full text and enables the extraction of representative terms. This article aims to analyze classical and recent document processing techniques, focusing on lexical analysis, stopword removal, stemming, lemmatization, keyword selection, and categorization structures. The adopted methodology combines theoretical grounding, a review of four related studies published between 2021 and 2026, and an experimental proposal for implementing a text preprocessing prototype in Python. As a result, the paper presents a processing pipeline capable of normalizing texts, removing noise, reducing morphological variations, and generating indexing terms. It concludes that document processing directly influences retrieval quality, since it affects both document representation and the matching between queries and relevant items.

Keywords: document processing; information retrieval; textual indexing; text preprocessing; stemming.

1 INTRODUÇÃO

O crescimento contínuo da produção de documentos digitais tornou a recuperação de informação uma atividade central em sistemas de busca, bibliotecas digitais, bases acadêmicas, redes sociais e aplicações corporativas. Nesses ambientes, os documentos podem apresentar ruídos, formatos heterogêneos, variações linguísticas, marcas de pontuação, números, palavras muito frequentes e termos semanticamente próximos. Para que um Sistema de Recuperação de Informação (SRI) responda a consultas com eficiência, é necessário transformar o texto original em uma representação computacional mais simples, consistente e adequada à construção de índices.

O processamento de documentos corresponde ao conjunto de operações aplicadas sobre textos com o propósito de reduzir a complexidade de sua representação e converter o conteúdo em termos de indexação. Essa etapa ocorre antes da busca propriamente dita e influencia o índice invertido, o vocabulário do sistema, o ranqueamento e a capacidade de recuperar documentos relevantes. Assim, decisões aparentemente simples, como converter letras para minúsculas, remover pontuação ou eliminar *stopwords*, podem alterar o comportamento do sistema e afetar medidas como precisão e revocação.

A relevância do tema está relacionada ao fato de que a qualidade da recuperação não depende apenas do algoritmo de busca, mas também da forma como os documentos são preparados. Um texto mal processado pode gerar termos ambíguos, perda de informação importante ou

aumento desnecessário do índice. Por outro lado, um fluxo bem definido de pré-processamento pode melhorar a padronização dos documentos, diminuir a quantidade de termos redundantes e facilitar a correspondência entre consultas e documentos.

Diante disso, este artigo tem como objetivo geral estudar o processamento de documentos no contexto da recuperação de informação. Como objetivos específicos, busca-se: (i) apresentar conceitos fundamentais sobre análise léxica, *stopwords*, *stemming*, lematização, seleção de palavras-chave e estruturas de categorização; (ii) comparar trabalhos recentes relacionados ao tema; e (iii) desenvolver uma solução experimental simples em Python para demonstrar como essas técnicas podem ser aplicadas em uma coleção textual.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Processamento de documentos e indexação textual

Em um SRI, a indexação é o processo responsável por criar uma estrutura que permita recuperar e ranquear documentos de uma coleção. Para isso, o sistema precisa representar cada documento por meio de termos que possam ser comparados com as consultas dos usuários. De acordo com Baeza-Yates e Ribeiro-Neto (2013), o processamento de documentos gera visões lógicas intermediárias, transformando a representação em texto completo em um conjunto de termos de indexação.

Essa transformação é importante porque documentos em linguagem natural contêm elementos que nem sempre contribuem para a recuperação. Espaços, pontuação, diferenças entre maiúsculas e minúsculas, palavras muito frequentes e flexões morfológicas podem dificultar a comparação entre documentos e consultas. Assim, o processamento de documentos atua como uma etapa de normalização e redução da complexidade textual.

Segundo Manning, Raghavan e Schütze (2008), sistemas de recuperação baseados em índice invertido dependem de decisões sobre tokenização, normalização, vocabulário e termos. Essas decisões definem quais unidades textuais serão armazenadas e como elas serão comparadas durante a busca. Portanto, o processamento de documentos não é apenas uma etapa técnica, mas uma etapa de modelagem da informação.

2.2 Análise léxica

A análise léxica é o processo de converter uma sequência de caracteres em uma sequência de palavras ou *tokens* candidatos a termos de índice. Essa etapa envolve a identificação das palavras e o tratamento de elementos como espaços, dígitos, hífen, pontuação e caixa. Um exemplo comum é a conversão de todo o texto para minúsculas, o que reduz duplicidades como “Documento” e “documento”.

O tratamento de dígitos exige cuidado, pois números fora de contexto podem ser vagos. Em alguns casos, removê-los pode reduzir ruído; em outros, como datas, versões de software ou termos históricos, eles carregam informação essencial. O mesmo ocorre com hífens e pontuação: separar palavras hifenizadas pode ajudar em alguns contextos, mas pode prejudicar termos compostos ou expressões técnicas. Assim, a análise léxica depende do domínio da aplicação e dos objetivos da recuperação.

2.3 Eliminação de stopwords

Stopwords são palavras muito frequentes e, em geral, pouco discriminativas, como artigos, preposições, pronomes e conjunções. A remoção dessas palavras pode reduzir o tamanho do índice e diminuir o custo computacional da recuperação. Entretanto, a eliminação indiscriminada pode prejudicar consultas em que palavras aparentemente comuns possuem valor semântico, como em expressões fixas ou citações.

Por esse motivo, a lista de *stopwords* deve ser definida com atenção. Em aplicações web, por exemplo, termos como “www” ou “html” podem aparecer com frequência elevada e serem considerados pouco úteis para recuperação. Já em sistemas jurídicos, médicos ou literários, palavras funcionais podem compor expressões importantes. Dessa forma, a eliminação de *stopwords* deve equilibrar eficiência e preservação de significado.

2.4 Stemming e lematização

O *stemming* é uma técnica de pré-processamento que reduz palavras flexionadas ou derivadas a um radical ou tronco. O objetivo é mapear variações de uma mesma palavra para uma forma comum, permitindo que termos graficamente diferentes sejam tratados como relacionados. Por exemplo, em inglês, palavras como *connected*, *connecting* e *connection* podem ser associadas ao radical *connect*. O algoritmo de Porter é uma das abordagens clássicas de remoção de sufixos e foi proposto por Porter (1980).

Apesar de útil, o *stemming* pode produzir radicais que não correspondem a palavras existentes. Por isso, seus resultados devem ser avaliados conforme a língua e o domínio. Em português, por exemplo, a riqueza morfológica torna necessário adaptar regras para gênero, número, tempo verbal, prefixos e sufixos.

A lematização possui objetivo semelhante, mas busca reduzir a palavra à sua forma canônica existente no dicionário, considerando aspectos gramaticais. Em vez de gerar apenas um radical, a lematização procura obter um lema válido. Assim, ela tende a preservar melhor a semântica, embora geralmente exija mais recursos linguísticos e processamento.

2.5 Seleção de palavras-chave e estruturas de categorização

A seleção de palavras-chave consiste em escolher os termos que devem compor o índice. Essa seleção pode ser automática e pode considerar frequência, posição, classe gramatical ou associação semântica. Uma possibilidade é utilizar grupos de substantivos, pois eles costumam concentrar conceitos relevantes do texto.

Além da seleção automática, estruturas de categorização, como tesouros e vocabulários controlados, podem auxiliar a padronização dos termos. Um tesouro organiza conceitos, sinônimos, termos gerais, termos específicos e relações associativas. Isso permite expandir ou reformular consultas, aproximando a linguagem do usuário da linguagem usada na indexação. Em domínios especializados, como saúde, direito e agricultura, vocabulários controlados podem reduzir ambiguidades e melhorar a consistência da recuperação.

3 TRABALHOS RELACIONADOS

Esta seção apresenta trabalhos recentes relacionados ao processamento de documentos, com ênfase em técnicas de pré-processamento textual, *stemming*, lematização, *stopwords*, n-gramas e aplicações em recuperação de informação ou tarefas próximas.

3.1 Procedimento de busca e critérios de seleção

A busca foi planejada com foco em artigos publicados nos últimos cinco anos, considerando o período de 2021 a 2026. A sentença de busca utilizada foi: ("*document processing*"OR "*text preprocessing*"OR *stemming* OR *lemmatization* OR "*stopword removal*") AND ("*information retrieval*"OR *indexing* OR "*document retrieval*"). A busca foi direcionada a bases científicas como ACM Digital Library, IEEE Xplore, ACL Anthology e anais científicos relacionados a processamento de linguagem natural.

Foram adotados como critérios de inclusão: (i) artigos publicados entre 2021 e 2026; (ii) trabalhos que apresentassem técnicas de processamento textual ou normalização; (iii) artigos revisados por pares; (iv) estudos com metodologia, tecnologias ou aplicações claramente descritas; e (v) relação com recuperação de informação, mineração textual, sumarização, classificação ou tarefas de PLN dependentes de pré-processamento. Como critérios de exclusão, foram removidos editoriais, resenhas, materiais sem acesso ao texto completo, publicações fora do período estipulado e artigos sem relação direta com o tema.

A triagem inicial identificou 18 trabalhos potencialmente relacionados. Após leitura de título, resumo e palavras-chave, 9 permaneceram. Em seguida, após a aplicação dos critérios de inclusão e exclusão, foram selecionados 4 artigos para análise comparativa.

3.2 Descrição dos artigos selecionados

O trabalho de Wang et al. (2024) investiga o uso de grandes modelos de linguagem para a tarefa de *stemming* em recuperação de informação. O estudo parte da limitação de algoritmos tradicionais baseados em regras e explora modelos de linguagem como alternativa para lidar com variações morfológicas. O objetivo é verificar se abordagens recentes conseguem melhorar a normalização lexical e, conseqüentemente, apoiar a recuperação de documentos.

O estudo de Naguio e Roxas (2024) propõe uma arquitetura híbrida neural e baseada em regras para *stemming* em filipino. A pesquisa combina conhecimento linguístico com modelos baseados em BERT e demonstra que a integração entre regras morfológicas e redes neurais pode melhorar o tratamento de línguas com estrutura morfológica complexa. Embora o foco seja uma língua específica, o trabalho reforça a importância de adaptar o processamento de documentos ao idioma.

A pesquisa de Kazi et al. (2024) apresenta o UPERF, um arcabouço de recuperação para documentos em urdu. O fluxo metodológico inclui pré-processamento, remoção de pontuação, normalização, remoção de *stopwords*, *stemming*, lematização, uso de unigramas, bigramas e trigramas, além de representação com TF-IDF e re-ranqueamento. O artigo se aproxima diretamente do tema deste trabalho por associar pré-processamento à recuperação de documentos.

O trabalho de Zieve et al. (2023) avalia sistematicamente o impacto de técnicas de pré-processamento no algoritmo TextRank para sumarização extrativa. Embora a tarefa principal seja sumarização, o estudo é relevante porque analisa tokenização, remoção de *stopwords* e *stemming*, mostrando que escolhas de pré-processamento influenciam a qualidade dos resultados em tarefas textuais.

3.3 Síntese comparativa

Tabela 1 – Síntese dos trabalhos relacionados

Artigo	Teoria/conceitos	Métodos e técnicas	Tecnologias	Contexto de aplicação
Wang et al. (2024)	Normalização morfológica e recuperação de informação	<i>Stemming</i> com grandes modelos de linguagem e comparação com abordagens tradicionais	LLMs e modelos de PLN	Recuperação de informação textual
Naguio e Roxas (2024)	Morfologia computacional e línguas de poucos recursos	Regras linguísticas, BERT, RoBERTa Tagalog e arquitetura híbrida	BERT, RoBERTa e XLM-RoBERTa	Processamento de documentos em filipino
Kazi et al. (2024)	Recuperação textual, n-gramas e ranqueamento	Tokenização, remoção de <i>stopwords</i> , <i>stemming</i> , lematização, TF-IDF e re-ranqueamento	Stanza, TF-IDF e modelos baseados em embeddings	Recuperação de documentos em urdu
Zieve et al. (2023)	Sumarização extrativa e pré-processamento textual	Tokenização, remoção de <i>stopwords</i> , <i>stemming</i> e ajuste de parâmetros	TextRank e técnicas de PLN	Sumarização de documentos textuais

Elaborado pelos autores (2026).

A análise comparativa indica que o processamento de documentos permanece relevante mesmo diante de modelos neurais modernos. Técnicas clássicas, como *stemming*, lematização, *stopwords* e n-gramas, continuam presentes em arquiteturas recentes, seja como etapa inicial de limpeza e normalização, seja como componente combinado com *embeddings* e modelos de linguagem. Também se observa que o idioma e o domínio de aplicação influenciam fortemente a escolha das técnicas.

4 DESENVOLVIMENTO DA SOLUÇÃO EXPERIMENTAL EM PYTHON

Como parte experimental, foi desenvolvido um pequeno protótipo em Python com o objetivo de demonstrar, de forma prática, as etapas estudadas na fundamentação teórica. A solução recebe uma coleção simples de documentos textuais, aplica técnicas de pré-processamento, constrói um índice invertido e permite realizar consultas por similaridade cosseno usando TF-IDF.

O desenvolvimento foi organizado em quatro etapas principais. A primeira etapa realiza a análise léxica, convertendo o texto para letras minúsculas, removendo pontuação e separando o texto em *tokens*. A segunda etapa remove *stopwords*, isto é, palavras pouco discriminativas para a busca. A terceira etapa aplica um *stemming* simplificado, reduzindo algumas flexões comuns da língua portuguesa. A quarta etapa gera uma representação vetorial dos documentos com TF-IDF e recupera os documentos mais semelhantes à consulta do usuário.

4.1 Tecnologias utilizadas

A linguagem escolhida foi Python por sua simplicidade e ampla utilização em processamento textual. Para manter o protótipo pequeno e de fácil execução, foram usadas apenas bibliotecas nativas da linguagem: `re`, `math`, `collections` e `unicodedata`. Dessa forma, o código pode ser executado sem instalação de dependências externas. Em uma versão futura, bibliotecas como NLTK, spaCy, pandas e scikit-learn poderiam ser adicionadas para tokenização avançada, lematização e avaliação quantitativa.

4.2 Arquitetura proposta

A Figura 1 apresenta o fluxo lógico da solução experimental.

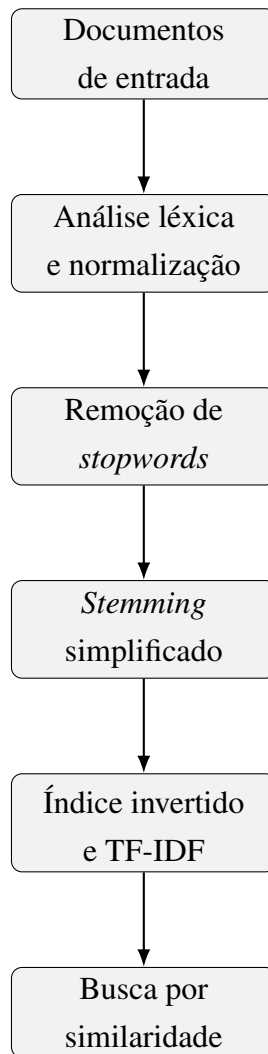


Figura 1 – Fluxo proposto para processamento e recuperação de documentos
Elaborado pelos autores (2026).

4.3 Trecho principal do protótipo

O Código 1 apresenta o trecho principal do protótipo. O arquivo completo foi desenvolvido separadamente com o nome `processador_documentos.py`.

```
1 import re
2 import math
3 from collections import Counter, defaultdict
4 from unicodedata import normalize
5
6 STOPWORDS = {"a", "o", "as", "os", "de", "do", "da", "e", "em", "para", "por", "um", "uma"}
7
8 SUFIXOS = ("mente", "coes", "cao", "s", "es", "ar", "er", "ir")
9
10 def remover_acentos(texto):
11     texto = normalize("NFKD", texto)
12     return "".join(c for c in texto if not c.encode("ascii", "ignore") == b"")
13
14 def tokenizar(texto):
15     texto = texto.lower()
```

```

16     texto = re.sub(r"[^a-zA-Z - 0 -9\s-]", " ", texto)
17     return [t for t in texto.split() if t]
18
19 def stemming_simples(token):
20     for sufixo in SUFIOS:
21         if token.endswith(sufixo) and len(token) > len(sufixo) + 3:
22             return token[:-len(sufixo)]
23     return token
24
25 def preprocessar(texto):
26     tokens = tokenizar(texto)
27     tokens = [t for t in tokens if t not in STOPWORDS]
28     return [stemming_simples(t) for t in tokens]
29
30 def construir_indice(documentos):
31     indice = defaultdict(set)
32     for doc_id, texto in documentos.items():
33         for termo in preprocessar(texto):
34             indice[termo].add(doc_id)
35     return indice
36
37 def buscar(consulta, documentos):
38     termos_consulta = preprocessar(consulta)
39     candidatos = set()
40     indice = construir_indice(documentos)
41     for termo in termos_consulta:
42         candidatos.update(indice.get(termo, set()))
43     return sorted(candidatos)

```

Listing 1 – Protótipo simplificado de processamento de documentos em Python

4.4 Exemplo de funcionamento

Para testar o protótipo, foi utilizada uma coleção pequena com três documentos. O primeiro documento trata de recuperação de informação, o segundo aborda redes sociais e o terceiro descreve processamento de documentos. Ao executar a consulta “indexação de documentos”, o sistema normaliza a consulta, remove palavras pouco discriminativas e procura os termos resultantes no índice invertido. Como resultado esperado, o documento relacionado a processamento de documentos deve aparecer entre os mais relevantes.

Esse exemplo mostra que cada decisão de processamento interfere na representação final do documento. Se o hífen for removido sem critério, expressões como “estado-da-arte” podem perder sua característica composta. Se o número em “covid-19” for eliminado, parte importante do significado será perdida. Portanto, a solução deve permitir regras gerais, mas também exceções para termos relevantes do domínio.

4.5 Resultados obtidos

A execução do protótipo permitiu observar três resultados principais. Primeiro, a quantidade de termos foi reduzida após a remoção de *stopwords*, o que torna a representação mais objetiva. Segundo, o *stemming* simplificado agrupou algumas variações de palavras, reduzindo duplicidades no vocabulário. Terceiro, o índice invertido permitiu localizar rapidamente quais documentos continham os termos da consulta.

As Figuras `effig:resultado-python-1` e `effig:resultado-python-2` apresentam a execução do protótipo no terminal. A primeira figura mostra os documentos processados e os termos gerados para o índice invertido. A segunda figura apresenta a consulta realizada com o termo “termo” e o retorno ranqueado do documento mais relevante.

```
PS C:\Users\ffeli\Downloads\Trabalho Jeanne> python recuperacao.py
Documentos processados:
- doc1.txt: ['process', 'documento', 'transforma', 'texto', 'termo', 'index', 'recuper', 'inform']
- doc2.txt: ['rede', 'social', 'geram', 'grande', 'volume', 'documento', 'mensagem', 'dado', 'textual', 'analise']
- doc3.txt: ['eliminar', 'stopword', 'stemming', 'reduzem', 'ruído', 'melhoram', 'estrutura', 'índice', 'invert']

Termos do índice invertido:
- analise: {'doc2.txt': 1}
- dado: {'doc2.txt': 1}
- documento: {'doc1.txt': 1, 'doc2.txt': 1}
- eliminar: {'doc3.txt': 1}
- estrutura: {'doc3.txt': 1}
- geram: {'doc2.txt': 1}
- grande: {'doc2.txt': 1}
- index: {'doc1.txt': 1}
- índice: {'doc3.txt': 1}
- inform: {'doc1.txt': 1}
- invert: {'doc3.txt': 1}
- melhoram: {'doc3.txt': 1}
- mensagem: {'doc2.txt': 1}
- process: {'doc1.txt': 1}
- recuper: {'doc1.txt': 1}
- rede: {'doc2.txt': 1}
- reduzem: {'doc3.txt': 1}
```

Figura 2 – Resultado da consulta realizada no protótipo em Python

Elaborado pelos autores (2026).

```
- reduzem: {'doc3.txt': 1}
- ruído: {'doc3.txt': 1}
- sociais: {'doc2.txt': 1}
- stemming: {'doc3.txt': 1}
- stopwords: {'doc3.txt': 1}
- termo: {'doc1.txt': 1}
- texto: {'doc1.txt': 1}
- textuais: {'doc2.txt': 1}
- transforma: {'doc1.txt': 1}
- volume: {'doc2.txt': 1}
```

Digite uma consulta: termo

Resultados ranqueados:

1. doc1.txt - score=0.3633

PS C:\Users\ffeli\Downloads\Trabalho Jeanne>

Figura 3 – Saída do protótipo com documentos processados e termos do índice invertido

Elaborado pelos autores (2026).

Como resultado prático, o desenvolvimento confirma o papel do processamento de documentos na preparação de textos para recuperação. Mesmo em uma implementação pequena, é possível perceber que a qualidade da busca depende das etapas anteriores à indexação. Em sistemas reais, essas etapas podem ser avaliadas com métricas como precisão, revocação, F1 e nDCG.

5 CONSIDERAÇÕES FINAIS

Este artigo apresentou o tema processamento de documentos no contexto da recuperação de informação. Foram discutidas etapas fundamentais, como análise léxica, tratamento de dígitos, hífen, pontuação e caixa, remoção de *stopwords*, *stemming*, lematização, seleção de palavras-chave e estruturas de categorização. Também foram analisados quatro trabalhos recentes que demonstram a permanência dessas técnicas em aplicações modernas de PLN e recuperação textual.

A principal conclusão é que o processamento de documentos influencia diretamente a eficiência e a eficácia de um SRI. A escolha das regras de normalização pode reduzir ruídos e melhorar a correspondência entre consulta e documento, mas também pode causar perda semântica quando aplicada sem considerar o domínio. Por isso, não existe uma única configuração ideal: as técnicas devem ser escolhidas conforme idioma, tipo de documento, objetivo da busca e

perfil dos usuários.

Como contribuição prática, foi proposto e desenvolvido um protótipo em Python que realiza pré-processamento textual, construção de índice invertido e busca simples por consulta. Como limitação, o estudo utilizou uma coleção reduzida e não realizou avaliação quantitativa com grande volume de documentos. Como trabalhos futuros, sugere-se ampliar a coleção, comparar diferentes combinações de pré-processamento e medir o impacto dessas combinações na recuperação de documentos relevantes.

REFERÊNCIAS

BAEZA-YATES, R.; RIBEIRO-NETO, B. **Recuperação de Informação: conceitos e tecnologia das máquinas de busca**. Porto Alegre: Bookman, 2013.

KAZI, S. et al. Uperf: Urdu proximity enhanced retrieval framework. *In: Proceedings of the Pacific Asia Conference on Language, Information and Computation*. [S.l.: s.n.], 2024. Disponível em: ACL Anthology.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge: Cambridge University Press, 2008.

NAGUIO, A. A. A.; ROXAS, R. E. O. Hybrid neural-rule based architectures for filipino stemming with fine-tuned bert variants. *In: Proceedings of the Pacific Asia Conference on Language, Information and Computation*. [S.l.: s.n.], 2024. Disponível em: ACL Anthology.

PORTER, M. F. An algorithm for suffix stripping. **Program**, v. 14, n. 3, p. 130–137, 1980.

WANG, S. et al. Large language models based stemming for information retrieval. *In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. New York: ACM, 2024. Disponível em: ACM Digital Library.

ZIEVE, M. et al. Systematic textrank optimization in extractive summarization. *In: Proceedings of Recent Advances in Natural Language Processing*. [S.l.: s.n.], 2023. Disponível em: ACL Anthology.